

DCCS: A dual congestion control signals based TCP for datacenter networks

**Project in computer networks - 236340
spring 2024**

Team: Ahmad Mhamed, Aia Younis, Amal Kassem

Advisor: Eran Tavor

Contents

Introduction	4
DCCS Overview	5
Algorithm	5
Ack Management	6
Dual Congestion Signals Management.....	7
Congestion Window Update Management	9
Parameters settings	11
Topology.....	13
Goals	14
Bonus Goal.....	14
Timetable	15
Protocols' Analysis	16
DCTCP	16
Fairness in DCTCP.....	16
Slow Convergence in DCTCP.....	17
DC-Vegas.....	18
Quick Convergence in DC-Vegas	18
Unfairness in DC-Vegas	19
DCCS	20
Fairness	20
Fast Convergence	20
Dynamic Adaptation.....	20
Evaluation	21
1. Fairness simulation	21
2. Convergence simulation	23
Challenges	26
• Delack and fairness	26
• Convergence and empty queues.....	27

• Large drain cycle problem	29
DCCS improvement	30
Future Work.....	31
Summary	32
Difficulties	32
Conclusions.....	32
Appendix.....	33
References	34

Introduction

Datacenter networks (DCNs) have evolved into critical infrastructures supporting diverse applications with demanding performance requirements. From cloud computing to real-time analytics and interactive gaming, these applications necessitate strict Quality of Service (QoS) guarantees, such as minimal latency, high throughput. The unique environment of DCNs characterized by rapid data flows, ultra-low latency, and the prevalence of congestion events poses significant challenges for achieving these requirements, highlighting the need for advanced congestion control mechanisms.

Traditional congestion control protocols, such as TCP Reno and Cubic, rely on coarse-grained congestion signals like packet loss, which are insufficient to address the complexities of DCNs. ECN-based protocols, such as DCTCP, improve congestion feedback granularity by marking packets when switch queues exceed a threshold. While effective in mitigating packet loss and queue buildup, they struggle with convergence issues in multi-bottleneck scenarios. Conversely, delay-based protocols like TCP Vegas focus on maintaining low latency but face fairness challenges due to inaccurate RTT estimations.

To overcome the shortcomings of ECN-based and delay-based protocols, Dual Congestion Control Signals (DCCS) is proposed, combining ECN and delay signals to achieve more precise and effective congestion management. DCCS leverages ECN to handle burst congestion effectively while utilizing delay signals to maintain low end-to-end latency. Additionally, it introduces a drain signal to enhance fairness by accurately estimating the minimum RTT for each flow.

Through extensive ns-3 simulations, we show that DCCS enhances fairness and achieves faster convergence. Our results highlight DCCS's potential to significantly enhance the performance and reliability of DCNs.

DCCS Overview

DCCS aims to attain optimal performance in terms of high throughput and low end-to-end delay across various DCN scenarios while maintaining resilience against traffic bursts, ensuring rapid convergence, and fostering fairness among competing flows.

To that end, it leverages the advantages provided by two congestion signals. It combines ECN and delay signals to:

- Manage burst congestion (using ECN).
- Minimize end-to-end delay (using delay signals).
- Handle cases of severe congestion by employing an EAD signal, combining ECN and delay mechanisms.

Furthermore, it introduces a drain signal to address the fairness issue present in delay-based protocols.

In this section, we will describe the protocol as presented in the paper [1].

Algorithm

DCCS is comprised of three primary modules:

1. Ack management module.
2. Dual congestion control signals management module.
3. Congestion window updates module.

When an ACK packet arrives at the sender, DCCS activates the ACK management module ([line 4](#)). This module evaluates the network queue length and separately records the number of packets

Algorithm 1: DCCS Algorithm

```
Input: ACK
1 Initialize: totalAck = 0, signal = 0 ;
2 foreach Arrived ACK do
3   totalAck = totalAck + ACK.size ;
4   /* congestion signal management */
5   ACKManagement(ACK) ;
6   if received all the ACKs of last window then
7     /* dual congestion signal management */
8     signal = DCSManagement() ;
9     /* congestion window management */
10    windowUpdate(signal) ;
11    reSet() ;
12  end
13 end
```

marked with RTT and ECN signals. Once all packets from the previous window have been acknowledged, DCCS moves to the

dual congestion signal management module (line 6), where it selects the appropriate congestion signal based on current network conditions. Next, the congestion window is adjusted (line 7), and finally, the packet counters (totalAck, ackedBytesDelay and ackedBytesECN) are reset to prepare for the next cycle.

Ack Management

- This module is initiated upon the arrival of an ACK packet at the sender.
- The purpose of this module is to detect congestion signals (ECN or delay).
- The Algorithm:
 - for each ACK packet:
 - Check whether ECN signal is triggered and tally number of ECN marked bytes.
 - Check whether ECN bit is set, indicating that the current queue length at the bottle-neck switch exceeds a predefined threshold K_{ECN} , signalling network congestion.
 - Check whether delay signal is triggered and tally number of delayed bytes.
 - Calculate the current RTT.
 - Estimate the current network queuing delay (NQ) using Eq. (1).
 - If NQ is greater than a predefined threshold K_{nq} , recognize the packet as a delay congestion signal.
 - Compute the average network queuing delay: $NQ_{Avg} = NQ_{avg} + \omega \times (NQ - NQ_{avg})$
 - ω is the weight used for exponentially averaging network.

Algorithm 2: ACK management algorithm

```

Input: ACK
Output: ackedBytesDelay, ackedBytesEcn
1 Initialize:  $NQ = 0$  ;
  /* check whether has ECN bit
2  $isECN \leftarrow hasECN(ACK)$  ;
3 if  $isECN$  then
4   |  $ackedBytesEcn += ACK.size$ ;
5 end
  /* network queue and the avg value
6  $RTT_c \leftarrow getRTT(ACK)$  ;
7  $updateMinRTT(RTT_c)$  ;
8  $NQ \leftarrow cwnd \times (RTT_c - RTT_{min}) / RTT_c$  ;
9 if  $NQ > K_{nq}$  then
10  |  $ackedBytesDelay += ACK.size$ ;
11 end
12  $NQ_{avg} \leftarrow NQ_{avg} + \omega \times (NQ - NQ_{avg})$ ;
  
```

- The [next module](#) will use NQ_{avg} to determine if an EAD signal should be activated.

Dual Congestion Signals Management

- This module is initiated when all the acks of the current window are received and (processed).
- The purpose of this module is to determine which congestion signal to activate.
- Delay signal
 - Purpose: To mitigate queuing delay and maintain a low end-to-end network delay even when there is no obvious congestion.
 - Activated when: A persistent queuing delay surpasses a predefined threshold (K_{NQ}) but without ECN-marked ACK packets. This means there is no obvious network congestion, but the accumulative queuing delay is high enough that it affects network performance.
- ECN signal
 - Purpose: To control burst traffic.
 - Activated when: When ECN-marked ACK packets arrive at the sender.
- EAD signal
 - Purpose: To effectively tackle severe network congestion occurring in multiple network bottlenecks while ensuring a low network delay.
 - Activated when: When the module perceives an ECN signal and the average network queuing delay exceeds a predefined threshold ($K_m = 1.5 \times K_{nq}$).
- Drain signal
 - Purpose: To allow a more accurate estimation of the minimum RTT (thus improving fairness).
 - Activated when: Periodically (see round and drain cycle in [Algorithm 3](#)).

- The Algorithm (see [Algorithm 3](#) for pseudocode):

- Steps:

1. Calculate ACK Fractions:

Compute the fraction of ACK packets marked with ECN and delay signals separately. Compute

$Alpha_{ECN}$ and $Alpha_d$ as an exponential moving average of the relevant fraction.

These values will be used by the [next module](#).

- $0 < g < 1$ is the smoothing factor.

([Lines 2–5](#)).

- These computations rely on information provided by the [previous module](#).

2. Check for Delay Signal: If only delayed ACK packets (without ECN markings) are observed ([Lines 6–8](#)).

3. Check for ECN Signal: If ECN-marked ACK packets are received ([Lines 9–11](#)).

4. Check for EAD Signal: If ECN-marked packets are observed *and* the average network queuing delay exceeds the threshold (K_m), the EAD signal is generated to address severe congestion across multiple bottlenecks ([Lines 12–14](#)).

5. Incorporate Drain Signal: Periodically empties the switch buffer to estimate the minimum RTT accurately and ensure transmission fairness ([Lines 15–19](#)).

- Signal Prioritization: Congestion signals are switched based on the observed scenario, with a strict priority ranking from lowest (delay signal) to highest (drain signal).

Algorithm 3: Dual congestion signals management algorithm

Input: $ackedBytesDelay$, $ackedBytesEcn$, NQ_{avg} , K_m
Output: $signal$

```

1 Initialize:  $Alpha_{ECN} = 0$ ,  $Alpha_d = 0$ ,  $ECNFrac = 0$ ,  $DelayFrac = 0$ ,  $signal = non\_sig$  ;
2  $ECNFrac \leftarrow ackedBytesEcn / totalAck$  ;
3  $Alpha_{ECN} \leftarrow (1 - g) \times Alpha_{ECN} + g \times ECNFrac$  ;
4  $DelayFrac \leftarrow ackedBytesDelay / totalAck$  ;
5  $Alpha_d \leftarrow (1 - g) \times Alpha_d + g \times DelayFrac$  ;
6 if  $DelayFrac > 0$  AND  $ECNFrac == 0$  then
7   |  $signal \leftarrow delay\_sig$  ;
8 end
9 if  $ECNFrac > 0$  then
10  |  $signal \leftarrow ecn\_sig$  ;
11 end
12 if  $ECNFrac > 0$  AND  $NQ_{avg} \geq K_m$  then
13  |  $signal \leftarrow ead\_sig$  ;
14 end
15  $round++ = 1$  ;
16 if  $round == drain\_cycle$  then
17  |  $signal \leftarrow drain\_sig$  ;
18  |  $round \leftarrow 1$  ;
19 end
20 return  $signal$  ;

```

Congestion Window Update Management

- This module is initiated upon the completion of the previous module i.e. once the appropriate signal is determined.
- The purpose of this module is to adjust the cwnd based on the active signal (determined by the previous module).
- The Algorithm (see [Algorithm 4](#) for pseudocode):

1. No Congestion Signal:

Increases the congestion window by one MSS during each RTT (standard additive increase mechanism).
([lines 2-5](#)).

2. ECN Signal:

Behaves similarly to DCTCP. The congestion window is updated using the formula:

$$cwnd = cwnd \times \left(1 - \frac{Alpha_{ECN}}{2}\right)$$

([lines 6-8](#)).

3. Delay Signal:

Employs a conservative scheme to eliminate network delay while preserving throughput.

The congestion window is updated using the formula:

$$cwnd = cwnd - (1 - Alpha_d)$$

([Lines 9-11](#))

4. EAD Signal:

Takes aggressive action to adjust the congestion window (to handle severe congestion).

The sender combines the ECN and delay congestion indicators ($Alpha_{ECN}$ and $Alpha_{delay}$) to form a

Algorithm 4: Window update algorithm

```

Input: signal, cwndin
Output: cwnd
1 cwnd  $\leftarrow$  0 ;
2 switch signal do
3   case non_sig do
4     cwnd  $\leftarrow$  cwndin + 1 ;
5   end
6   case ecn_sig do
7     cwnd  $\leftarrow$  cwndin  $\times$  (1 -  $Alpha_{ECN}/2$ );
8   end
9   case delay_sig do
10    cwnd  $\leftarrow$  cwndin - (1 -  $Alpha_d$ );
11  end
12  case ead_sig do
13     $Alpha_{EAD} = Alpha_{ECN} + Alpha_d$  ;
14    cwnd  $\leftarrow$  cwndin  $\times$  (1 -  $Alpha_{EAD}/2$ );
15  end
16  case drain_sig do
17    cwnd  $\leftarrow$  min(cwndin, drain_cwnd) ;
18  end
19 end
20 return cwnd ;

```

new congestion indicator ($Alpha_{EAD}$):

$$Alpha_{EAD} = Alpha_{ECN} + Alpha_d$$

Subsequently, the congestion window is updated similarly to DCTCP:

$$cwnd = cwnd \times \left(1 - \frac{Alpha_{EAD}}{2}\right)$$

([Lines 12-15](#))

5. Drain Signal:

Upon receiving the drain signal, the sender should decrease its congestion window to avert network queuing. This recurring drain signal permits flows to perceive their minimum RTTs accurately, thereby promoting fairness.

The congestion window is updated using the formula: ¹

$$cwnd = \min(cwnd_{in}, cwnd_{drain})$$

Where $cwnd_{drain}$ is a predetermined value.

([Lines 16-18](#))

¹ We changed this formula in challenge [3.a](#).

Parameters settings

Our implementation of DCCS relies on several critical parameters to achieve efficient congestion management and fairness across multi-bottleneck topologies. These parameters are:

K_{ECN} , K_{nq} , K_m , *drain cycle* and *drain cwnd*.

For K_{ECN} and K_{nq} , we adopt the default values proposed in prior works [2,3], as these have been extensively validated. Our primary focus is on determining K_m , *drain cycle* and *drain_cwnd* to optimize performance under varying network conditions.

Setting K_m

K_m serves as a threshold to detect significant network congestion, particularly in multi-bottleneck scenarios. Ideally, K_m should reflect twice the congested queue length, but in practice, the instantaneous queue length is inaccessible. Instead, we leverage the average queue length NQ_{avg} over an RTT period. To accommodate burst traffic, we define $K_m = 1.5 * K_{nq}$, ensuring that the Early Alert Detection (EAD) signal is triggered when $K_m \leq NQ_{avg}$.

This approach enables the system to manage bursty traffic while maintaining stable performance.

Configuring *drain_cwnd*

To accurately estimate the minimum RTT, the DCCS protocol reduces the congestion window to *drain_cwnd* at regular intervals defined by *drain_cycle*. The value of *drain_cwnd* is chosen to ensure that the total flow throughput remains below the bottleneck link's capacity, as described by:

$$\sum_{i=0}^N cwnd(i) \leq RTT * C$$

where C is the link capacity, N is the number of flows, and $cwnd(i)$ is the congestion window for flow i . For optimal operation, *drain_cwnd* is set as:

$$drain_wnd = \max\left(\left\lfloor \frac{RTT * C}{N} \right\rfloor, W_{init}\right)$$

where W_{init} represents the initial window size.

Choosing drain_cycle

The parameter drain_cycle determines the interval for triggering the drain_signal. It balances throughput and responsiveness by allowing enough time to probe the network's minimum RTT without excessively reducing the congestion window. Based on practical considerations mentioned in the paper, drain_cycle was defined as:

$$drain_{cycle} = RTT * C + B_{\Delta} - W_{init}$$

Here, B_{Δ} represents the additional buffer size tolerated without triggering congestion signals. Typically, B_{Δ} is set equal to K_{nq} .

Practical Considerations

- Obtaining accurate values for RTT, C, W_{init} , and B_{Δ} is crucial. In our experiments, C is defined as the bandwidth of the edge switch link, and W_{init} is fixed at 10 packets.
- drain_cwnd is multiplied by 0.6.
- The drain_cycle and the drain_cwnd are not constantly adjusted, instead, we initialize them at first and after drain_cycle we adjust them once and then we leave them unchanged.

Topology

To measure fairness across different protocols, we utilized a widely used multi-bottleneck topology with three concurrent flows: TR1, TR2, and TR3. This setup allowed us to effectively assess fairness by evaluating protocols such as DCTCP, DC-Vegas and DCCS. Once the network reaches convergence, we calculate Jain's fairness index for each flow.

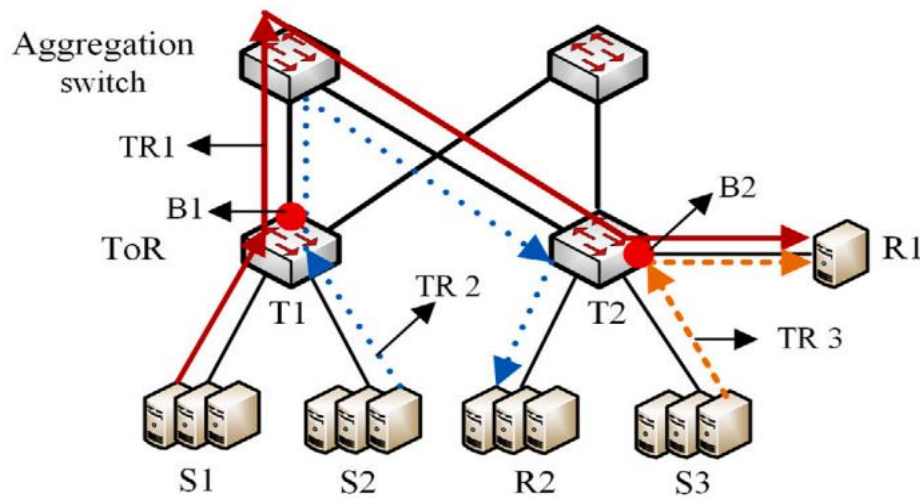
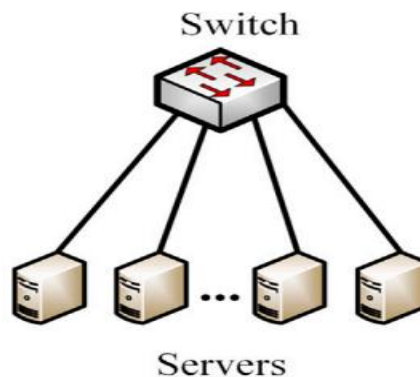


Fig. 3. Multi-bottleneck topology.

In addition, for simplicity and a streamlined performance analysis, we used a single-rack topology to assess convergence.



Goals

Implementation of protocols & topologies

- Implement DCCS, DCTCP, and DC-Vegas protocols.
- Set up the network topology used for analysing.

Validation of Issues in Existing Protocols

- Demonstrate the fairness issue inherent in DC-Vegas.
- Verify the convergence issue in DCTCP.

Evaluation of DCCS Performance

- Show that DCCS improves fairness over DC-Vegas.
- Confirm that DCCS enhances convergence compared to DCTCP

Bonus Goal

Improve DCCS.

Timetable

Week 1: introduction to the project.

Week 2-3: Research and learn:

- Learn about congestion control.
- Learn about different congestion signals and protocols.
- Read the paper.
- Learn how to use ns-3.

Week 4: CDR presentation

Week 5-6:

- Implement multi-bottleneck topology.
- Run existing implementation of DCTCP on the topology to evaluate its fairness.

Week 7-9:

- Implement DC-Vegas protocol.
- Run DC-Vegas on the implemented multi-bottleneck topology to evaluate its fairness and demonstrate its limitations.

Week 10-12:

- Implement single-rack topology.
- Add traces to the simulation to monitor convergence and evaluate the convergence time for DCTCP and DC-Vegas.

Week 13-15:

- Implement DCCS protocol.
- Run DCCS on multi-bottleneck topology and evaluate its fairness.
- Run DCCS on single-rack topology and evaluate its convergence.

Protocols' Analysis

DCTCP

Datacenter TCP (DCTCP) serves throughout the paper as a representative of ECN-based congestion protocols.

It achieves fairness but has a slower convergence rate due to the design of its congestion control mechanism.

Fairness in DCTCP

- ECN-Based Congestion Control:
 - DCTCP uses ECN to detect congestion without relying on packet loss. ECN provides a real-time, accurate signal of congestion based on queue occupancy.
 - This mechanism allows DCTCP to maintain fairness across multiple competing flows by ensuring all flows respond to the same congestion signal consistently.
- Fractional Window Reduction:
 - DCTCP reduces the congestion window proportionally to the fraction of ECN-marked packets ($Alpha_{ECN}$):
$$cwnd = cwnd \times \left(1 - \frac{Alpha_{ECN}}{2}\right)$$
 - This fine-grained adjustment ensures that each flow reduces its rate proportionally, preventing unfair dominance by larger or smaller flows.

Slow Convergence in DCTCP

- Gradual Window Adjustments:
 - The congestion window in DCTCP is reduced proportionally to α_{ECN} , which depends on the fraction of ECN-marked packets.
 - This gradual reduction slows the convergence rate compared to protocols that use more aggressive adjustments (e.g., halving the window as in TCP Reno).
- Persistent Queuing Delays:
 - Due to its fine-grained adjustments, DCTCP may take longer to fully drain queues, especially when multiple flows compete. This can delay reaching the optimal operating point.
- ECN Fraction Estimation Lag:
 - The calculation of α_{ECN} uses an exponential moving average of ECN-marked packets, introducing a lag in the protocol's reaction to changes in network conditions.
 - This smoothing effect, while stabilizing the protocol, contributes to slower convergence.

DC-Vegas

DC-Vegas serves throughout the paper as a representative of delay-based congestion protocols.

It calculates the current network queue length, denoted as Q_c , using [Eq. \(1\)](#) and then compares Q_c with a threshold, represented by K_{dcv} . Then, the senders measure the proportion of packets for which Q_c exceeds K_{dcv} within each RTT ([Eq. \(2\)](#)).

A delay indicator (α_{dcv}) is calculated with [Eq. \(3\)](#).

Finally, the senders adjust the congestion window (cwnd) in accordance with

$$\begin{aligned} 1) \quad Q_c &= \frac{cwnd}{RTT_e} \times (RTT_c - RTT_{min}) \\ 2) \quad F_{dcv} &= \frac{\text{number of ACKs with } Q_c > K_{dcv}}{\text{Total number of ACKs in a window}} \\ 3) \quad \alpha_{dcv} &= (1 - g) \times \alpha_{dcv} + g \times F_{dcv} \\ 4) \quad cwnd &= \begin{cases} cwnd \times \left(1 - \frac{\alpha_{dcv}}{2}\right) & \text{If } F_{dcv} > 0 \\ cwnd + 1 & \text{If } F_{dcv} = 0 \end{cases} \end{aligned}$$

the congestion level ([Eq. \(4\)](#)).

This protocol is known for its fast convergence but suffers from unfairness among competing flows.

Quick Convergence in DC-Vegas

- Delay-Based Congestion Detection:
 - DC-Vegas uses queuing delay as the primary congestion signal, allowing it to detect congestion before packet loss or ECN marks occur.
 - This enables the protocol to react quickly to changes in network conditions, ensuring rapid convergence to a stable state.
- Aggressive Window Adjustment:
 - DC-Vegas adjusts the congestion window based on estimated queuing delays.

- This adjustment happens immediately upon detecting delay changes, allowing the protocol to converge faster than ECN- or loss-based protocols, which rely on delayed feedback.
- Minimal Smoothing:
 - DC-Vegas often employs minimal smoothing of delay signals, enabling quicker adaptation to real-time network conditions.

Unfairness in DC-Vegas

- Dependence on Minimum RTT:
 - DC-Vegas uses the minimum RTT (RTT_{min}) as a baseline to estimate queuing delays: $\text{queuing delay} = RTT_c - RTT_{min}$
(RTT_c represents the currently sampled RTT and RTT_{min} represents the minimum sampled RTT since the connection is established.)
 - Flows with larger RTT_{min} inherently estimate smaller queuing delays for the same RTT_c , leading them to perceive less congestion.
 - These flows experience fewer reductions in their congestion window, allowing them to consume more bandwidth unfairly compared to flows with smaller RTT_{min} .
- Latecomer Advantage:
 - Newer flows often estimate a larger RTT_{min} because existing flows have already filled the queue. This latecomer advantage allows newer flows to achieve higher throughput, further exacerbating unfairness.
- Sensitivity to Network Topology:
 - Delay-based protocols like DC-Vegas are sensitive to path variations and bottleneck distances. Flows traversing shorter paths with smaller RTT_{min} are penalized compared to longer-path flows.

DCCS

The Dual Congestion Control Signals (DCCS) protocol achieves a balance between fairness (a strength of ECN-based protocols) and fast convergence (a strength of delay-based protocols). It combines the advantages of both approaches while mitigating their individual weaknesses.

Fairness

- Mitigation of RTT Bias:
 - DCCS drains the queue buffers periodically (drain signal), allowing for more accurate estimates of RTT_{min} .
 - Unlike delay-based protocols (e.g., DC-Vegas), which rely heavily on RTT_{min} , DCCS mitigates unfairness caused by RTT variations by incorporating ECN as an independent congestion indicator.

Fast Convergence

- Delay Signal Integration:
 - DCCS uses queuing delay to detect congestion earlier than ECN, enabling quick reactions to changes in traffic conditions.
- Combined Indicators (EAD Signal):
 - In severe congestion scenarios, DCCS combines ECN and delay signals into the EAD signal, allowing aggressive and efficient adjustments to the congestion window for rapid stabilization.

Dynamic Adaptation

- Signal Switching:
 - DCCS dynamically switches between ECN, delay, EAD, and drain signals based on current network conditions.
 - This ensures that the protocol adapts to varying congestion scenarios while maintaining fairness and convergence.

Evaluation

We employed ns-3 simulations to evaluate whether DCCS can rapidly converge to fairness. Additionally, we demonstrated the fairness issue in DC-Vegas and the convergence challenges in DCTCP.

To do this, we conducted two different simulations:

1. Fairness simulation:

In this simulation, we evaluated the fairness of DCTCP, DC-Vegas, and DCCS using the [multi-bottleneck topology](#) described in the topology section. All links in the topology were configured with a bandwidth of 10 Gbps to ensure uniform link capacity and consistent evaluation conditions across the network.

The simulation involved three groups of traffic flows:

- Traffic 1 (TR1): 50 flows from S1 to R1 (50 senders and one receiver).
- Traffic 2 (TR2): 30 flows from S2 to R2 (30 senders and 30 receivers).
- Traffic 3 (TR3): 100 flows from S3 to R1 (100 senders and one receiver).

To simulate realistic data transfer, we used TCP with an ON-OFF behaviour, using On-Off application traffic pattern.

TR1, TR2, and TR3 were initiated asynchronously within a 0.5-second window, we collected throughput data for all flows and computed Jain's Fairness Index between 1.5 and 2 seconds after the start of the flows. This interval allowed the flows to stabilize, providing a consistent basis for measuring fairness. Jain's Fairness Index was used to quantify how equitably bandwidth was distributed among the flows, with values closer to 1 indicating higher fairness.

Additionally, we calculated the average throughput for each flow in the three traffic groups to assess how bandwidth was distributed among competing flows.

Results:

- DCTCP: we ran the DCTCP protocol on the previously described topology and simulation setup. Since DCTCP requires specific configurations, such as ECN marking at the switches, we set the ECN marking threshold to 50 packets. With this setup, DCTCP achieved highly satisfactory fairness results, as anticipated

```
1 Fairness for S1-R1 flows: 0.997
2 Fairness for S2-R2 flows: 0.999
3 Fairness for S3-R1 flows: 1.000
4 Aggregate user-level throughput for flows through T1: 9.717 Gbps
5 Aggregate user-level throughput for flows to R1: 9.717 Gbps
```

- DC-Vegas: we ran DC-Vegas protocol on the same topology and simulation setup. Our results demonstrated poor fairness among flows, as the protocol struggled to balance the allocation of bandwidth equitably across the network, due to inaccurate estimation of RTT minimum as explained in the introduction section above.

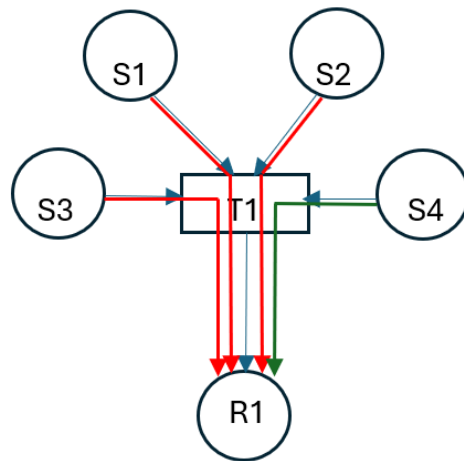
```
1 Fairness for S1-R1 flows: 0.391
2 Fairness for S2-R2 flows: 0.844
3 Fairness for S3-R1 flows: 0.811
4 Aggregate user-level throughput for flows through T1: 9.701 Gbps
5 Aggregate user-level throughput for flows to R1: 9.713 Gbps
```

- DCCS: We ran DCCS on the same topology with the appropriate configuration. The results showed that DCCS achieved good fairness, with a balanced distribution of bandwidth across all flows.

```
1 Fairness for S1-R1 flows: 0.939
2 Fairness for S2-R2 flows: 1.000
3 Fairness for S3-R1 flows: 0.997
4 Aggregate user-level throughput for flows through T1: 9.717 Gbps
5 Aggregate user-level throughput for flows to R1: 9.717 Gbps
```

2. Convergence simulation:

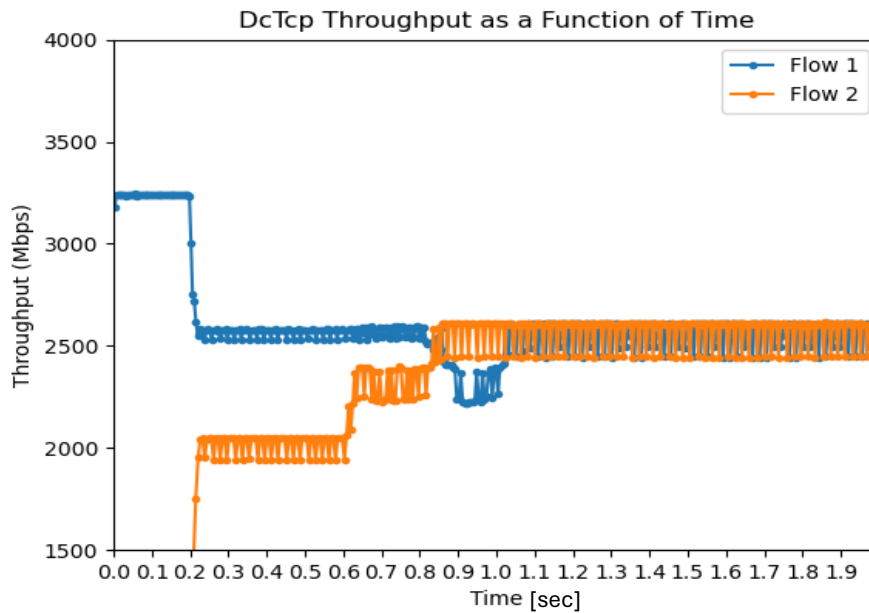
To thoroughly investigate the convergence properties of DCTCP, DC-Vegas, and DCCS, we built a simulation using a [single-rack topology](#). The setup consisted of 4 senders and 1 receiver, connected via a 10 Gbps link. We initiated 4 flow scenarios where 3 flows (Flow 1, Flow 2, and Flow 3) started simultaneously, and a subsequent flow (Flow 4) entered the network after a 0.2-second of delay.



The staggered start of Flow 4 was designed to create a transient imbalance in the network, simulating a dynamic scenario where a new flow competes for bandwidth with established flows. By analysing how quickly and effectively the protocols redistributed bandwidth among the flows, we could evaluate their convergence behaviour. To measure convergence, we monitored the throughput of each flow over time and draw throughput as a function of the time to find out how much it took for the system to stabilize, we will present the throughput of flow 1 and flow 4, since the 2nd and 3rd flow are similar to the 1st one.

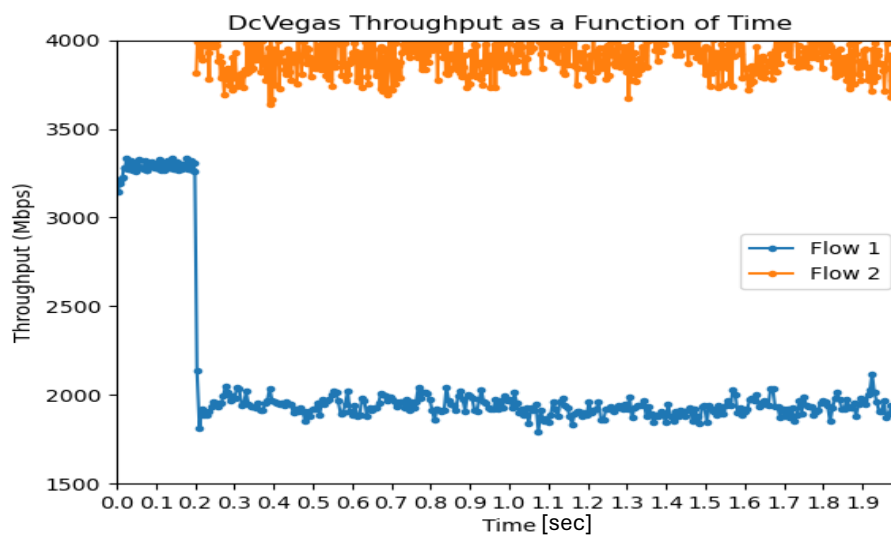
Results:

- DCTCP:



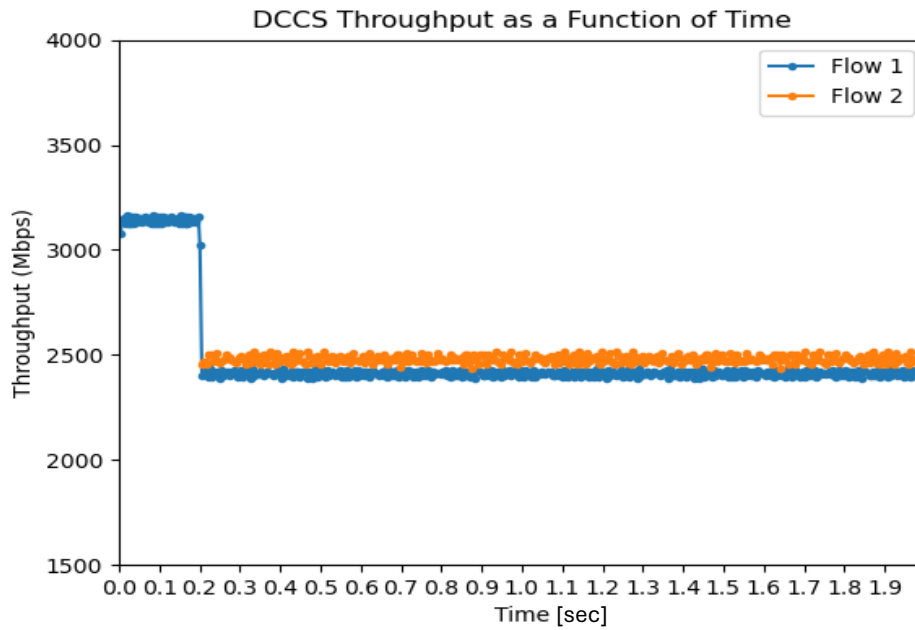
It took 0.8 seconds for DCTCP to stabilize, during which it successfully converged to a fair state.

- DC-Vegas:



DC-Vegas converged almost instantly, but to an unfair state.

- DCCS:



There is a significant improvement in convergence time compared to DCTCP, along with a substantial increase in fairness compared to DC-Vegas.

Challenges

We encountered several challenges during our work, and we will highlight the most significant ones:

- **Delack and fairness**

We ran the pre-existing DCTCP implementation on the [multi-bottleneck topology](#) described in earlier sections and received surprising results:

```
1 Fairness for S1-R1 flows: 1.000
2 Fairness for S2-R2 flows: 0.998
3 Fairness for S3-R1 flows: 0.492
4 Aggregate user-level throughput for flows through T1: 9.617 Gbps
5 Aggregate user-level throughput for flows to R1: 9.506 Gbps
```

Contrary to our expectations, DCTCP was not fair.

Some flows in S3 and all flows in S1 had a much lower throughput than expected and suffered compared to other flows.

Upon closer examination, we found that these flows (that had lower throughputs) received an ACK packet every ~5 ms, which is the RTO_{min} . When we examined flows with low throughput (for example flow 0 in S3) we found that an ack packet was received each ~5milli seconds which is RTO_{min} , so we thought about the possibility of Delack being the problem.

Delack or Delayed ACK is a mechanism that defers sending acknowledgments for received packets, instead of sending an ack for every received packet, the receiver waits and sends one ack for several received packets at once.

In the general setting, this mechanism helps reduce network overhead. In our case, using delack might affect the accuracy and responsiveness of our protocols.

Upon a closer examination of delack in ns-3, we found that it is set by default to 2. This caused the receivers in our simulations to send ACKs only after a timeout when the congestion window was sufficiently decreased (when $cwnd = 1$).

In these cases, the receiver would receive a packet but would not send an ACK due to the delayed ack mechanism. Instead, it would wait for a second packet to arrive.

The sender on the other hand, will not send a second packet before receiving an ACK (his congestion window is 1).

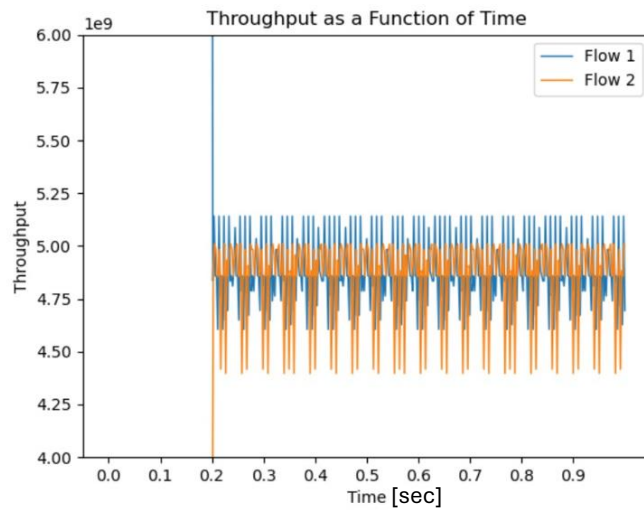
As such, an ACK will be sent (and received) only every RTO_{min} , which will cause the senders to not increase their congestion windows.

Changing the parameter to 1 "fixed" the fairness issue we found with DCTCP and the results we received met our expectations (see [the evaluation](#) section for the results).

- **Convergence and empty queues**

In the evaluation part, we aimed to replicate a similar simulation to the one described in the paper, where two senders and one receiver were used. We initiated two flow scenarios over a 10 Gbps link, with one flow (Flow 1) instantly capturing the full link bandwidth upon activation, and a subsequent flow (Flow 2) entering after a certain delay. We expected congestion to arise following the activation of Flow 2. However, we observed that the switch never became congested, consistently maintaining at most one packet in its buffer. For some reason, two senders were insufficient to create congestion, even with the limited 10 Gbps bandwidth between the switch and the receiver. This led us to adjust the simulation as described in the evaluation section above.

Results we got from running DCTCP on our first simulation:



The convergence time was nearly zero, which seemed unreasonable. Several factors could have contributed to these unexpected results, including issues in the simulation setup or parameter configuration. Despite our efforts to identify the root cause, we couldn't arrive at a definitive explanation. To address this, we added two more senders to the setup, which successfully induced the necessary congestion and allowed us to proceed with our evaluation.

- Large drain cycle problem

During the implementation of DCCS, we encountered unexpected results. Upon debugging, we discovered that the `drain_cycle` was incorrectly multiplied by the segment size. This miscalculation caused the drain cycle to become excessively large. As a result, the drain signal was not triggered, leading to inaccurate estimations of the minimum RTT (`RTT_min`). This persistent inaccuracy adversely affected fairness, resulting in degraded performance.

The mistake occurred because several parameters in the implementation, such as `drain_cwnd`, were in byte units and required multiplication by the segment size for proper calculations. Due to the similarity between `drain_cycle` and these parameters, we mistakenly applied the same operation to `drain_cycle`, leading to the issue.

This is what we observed while debugging:

```
Progress to 0.1 seconds simulation time
Progress to 0.2 seconds simulation time
Progress to 0.3 seconds simulation time
Progress to 0.4 seconds simulation time
Progress to 0.5 seconds simulation time
Progress to 0.6 seconds simulation time
Progress to 0.7 seconds simulation time
Progress to 0.8 seconds simulation time
Progress to 0.9 seconds simulation time
Progress to 1.0 seconds simulation time
Progress to 1.1 seconds simulation time
Progress to 1.2 seconds simulation time
Progress to 1.3 seconds simulation time
Progress to 1.4 seconds simulation time
Progress to 1.5 seconds simulation time
Progress to 1.6 seconds simulation time
Progress to 1.7 seconds simulation time
Progress to 1.8 seconds simulation time
Current CWND: 36500      Drain CWND: 7584
Current CWND: 32120      Drain CWND: 7617
Current CWND: 30660      Drain CWND: 7651
Progress to 1.9 seconds simulation time
Progress to 2.0 seconds simulation time
```

DCCS improvement

Drain congestion window

We found [line 17](#) of Algorithm 4:

$$cwnd = \min(cwnd_{in}, cwnd_{drain})$$

to be problematic.

We encountered problems because $cwnd_{drain}$ was always greater than $cwnd_{in}$, even when $cwnd_{in}$ was too large (the network queues were not drained properly).

To rectify this problem, we changed this line to:

$$cwnd = \min\left(cwnd_{in}, \frac{cwnd_{drain}}{n}\right)$$

Where n is the number of flows.

In the paper, they assumed that $n = 1$ and analyzed the extreme case where there is only one flow.

We relied on their general analysis but did not assume that $n = 1$.

Slow start threshold

A larger ssthresh interfered with the effectiveness of the drain signal. It allowed the cwnd to increase significantly (after it was reduced in response to the drain signal) before the network queues were drained.

To resolve this issue, we determined the value of ssthresh based on the drain congestion window:

$$cwnd < 10 \Rightarrow ssthresh = 10$$

This ensured that the slow start threshold was never large enough to interfere with draining the network queues.

Future Work

We doubt the correctness of [line 10](#) in algorithm 4.

If a delay signal is activated, then the congestion window is updated based on the following formula:

$$cwnd = cwnd_{in} - (1 - Alpha_d)$$

We would expect the congestion window to be reduced more aggressively in cases of severe delay, but that is not the case since $Alpha_d$ is a delay indicator that can range from 0 (no delay) to 1 (severe delay).

We tried to update the congestion window differently:

$$cwnd \leftarrow cwnd - (1 + Alpha_d) \text{ or } cwnd \leftarrow cwnd \times \left(1 - \frac{Alpha_d}{2}\right)$$

In these updates, the greater $Alpha_d$'s value, the greater the reduction that the congestion window will sustain.

Even with our changes, we did not observe significant changes in the results. We assume that this is because this signal is not always used by the protocol and that cases of severe delay are likely to be handled by the EAD signal. We believe more research is needed to better understand the scenarios in which the delay signal is activated, the impact and validity of the current window update mechanism, and how it might be optimized.

Summary

Difficulties

- While NS-3 is a powerful (and useful) tool, it also has a steep learning curve. Many of the difficulties we faced throughout this project were due to misunderstanding or being ignorant of details related to ns-3.
- We found that the paper omitted many details relevant to the simulations and that the pseudocode provided was not well written in some cases.

Conclusions

In this project, we implemented and evaluated DCTCP, DC-Vegas, and DCCS on different topologies.

Our results showed that:

- DCTCP exhibited good fairness but struggled with convergence.
- DC-Vegas faced fairness issues, leading to inconsistent performance.
- DCCS effectively improved both fairness and convergence, outperforming the other protocols in these aspects.

These insights demonstrate the advantages of DCCS in managing network congestion and providing a more balanced and efficient approach to network performance.

In our project, we managed to complete all our goals successfully and to improve the implementation of DCCS.

Appendix

- See <https://github.com/a-kassem/DCCS> for our code and traces.
- All code relevant to the single-rack topology can be found in "scratch/Convergence".
 - To replicate the issue described in [dccc improvement](#), see "dccc_conv_without_flow_num".
 - The directory "defective_dccc_conv" includes the code relevant to the [large drain cycle problem](#).
- All code relevant to the multi-bottleneck topology can be found in "scratch/Fairness".
 - To replicate the issue described in challenge [1](#), see "defective_dctcp_fairness".
- "src/internet/model" contains the source code and headers of the protocols.
- We recommend the following workflow:
 1. Set up ns-3 based on their official instructions.
 2. Clone our repository.
 3. Recursively copy our directory into ns-allinone-3.40/ns-3.40/ (the directory that contains scratch and src directories).
 4. Add the relevant protocols to "src/internet/CMakeLists.txt".
 - For each protocol, you must add the relative paths of both the .cc and .h files.

References

- [1] X. M. C. C. Yifei Lu, "DCCS: A dual congestion control signals based TCP for datacenter networks," *Computer Networks*, 2024. <https://doi.org/10.1016/j.comnet.2024.110457>.
- [2] A. G. D. M. J. P. P. P. B. P. S. S. M. S. M. Alizadeh, "Data center TCP (DCTCP)," *ACM SIGCOMM Computer Communication Review*, 2010.
<http://dx.doi.org/10.1145/1851275.1851192>.
- [3] J. W. C. L. Z. X. Y. H. J. Wang, "J. Wang, J. Wen, C. Li, Z. Xiong, Y. Han, DC-vegas: A delay-based TCP congestion control algorithm for datacenter applications," *Journal of Network and Computer Applications*, 2015.
<http://dx.doi.org/10.1016/j.jnca.2015.03.010>.